

2D Retro Games: Game Agent Training with PPO

Saketh

IMT2022085

Vandana

IMT2022123

Index Terms—PPO, Gymnasium, 2D Game agent training, Gym-retro, stable-retro, stable-baselines

I. INTRODUCTION

We have explored different tools and paradigms for game agent training for 2D retro games. Out of the different reinforcement learning methods, we deemed PPO to be our method of choice, with its monotonic policy improvement capabilities. We have trained game agents to play these games:

- AirStriker Genesis
- Mortal Kombat II(Genesis)

II. PRELIMINARIES

A. Proximal Policy Optimization Algorithms

Proximal Policy Optimization (PPO) [5] is a model-free, on-policy reinforcement learning algorithm introduced by Schulman et al. in 2017. It aims to combine the stability of Trust Region Policy Optimization (TRPO) [4] with the simplicity and efficiency of first-order optimization methods. PPO achieves this by introducing a clipped surrogate objective function, allowing multiple epochs of minibatch updates without the need for complex second-order methods.

• Policy Gradient Background

Policy gradient methods optimize the expected return by adjusting policy parameters θ in the direction of the gradient of the expected reward. The standard objective is

$$L^{PG}(\theta) = \mathbb{E}_t \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right],$$

where $\hat{A}_t$ is an estimator of the advantage function at time t . However, performing multiple gradient updates on the same data can lead to large policy updates, potentially destabilizing training.

• Clipped Surrogate Objective

To address the instability of large policy updates, PPO introduces a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio between the new and old policies, and ϵ is a hyperparameter (typically 0.1 or 0.2). This objective penalizes changes to the policy that move $r_t(\theta)$ outside the interval $[1 - \epsilon, 1 + \epsilon]$, thus preventing large updates that could destabilize training.

• Algorithmic Implementation

The PPO algorithm alternates between sampling data through interaction with the environment and optimizing the policy using the clipped surrogate objective. The steps are as follows:

- **Data Collection:** Run the current policy π_{θ_k} in the environment to collect a set of trajectories $\mathcal{D}_k$.
- **Advantage Estimation:** Compute advantage estimates $\hat{A}_t$ using methods like Generalized Advantage Estimation (GAE).
- **Policy Update:** Optimize the policy by maximizing $L^{CLIP}(\theta)$ using stochastic gradient ascent (e.g., with the Adam optimizer) over multiple epochs and minibatches.
- **Value Function Update:** Simultaneously, update the value function parameters ϕ by minimizing the squared-error loss:

$$L^{VF}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{R}_t \right)^2 \right],$$

where $\hat{R}_t$ is the empirical return.

• Relation to TRPO

While TRPO enforces a hard constraint on the KL divergence between the new and old policies, PPO approximates this constraint by clipping the probability ratio in the objective function. This approach simplifies the optimization process by avoiding second-order methods and allows for more straightforward implementation and tuning.

• Advantages and Limitations

Advantages:

- **Simplicity:** PPO is easier to implement compared to TRPO, as it avoids complex second-order optimization and constraints.
- **Stability:** The clipped objective prevents large policy updates, contributing to stable and reliable training.
- **Sample Efficiency:** PPO allows multiple epochs of minibatch updates, improving sample efficiency over standard policy gradient methods.
- **Versatility:** PPO performs well across a range of tasks, including continuous control and discrete action environments.

Limitations:

- **On-Policy Requirement:** PPO requires fresh data from the current policy for each update, which

can be less sample-efficient compared to off-policy methods.

- **Hyperparameter Sensitivity:** Performance can be sensitive to the choice of hyperparameters, such as the clipping parameter ϵ and learning rate.

III. LIBRARIES

A. *gym-retro*

Overview

gym-retro [1] is an open-source Python library developed by OpenAI that enables reinforcement learning research on a wide variety of classic video games. It provides a standardized interface to hundreds of games from game consoles such as Sega Genesis, NES, and SNES, allowing agents to interact with these environments using the familiar OpenAI Gym API. *gym-retro* supports features like save states, frame skipping, and reward customization, making it a powerful tool for benchmarking RL algorithms on complex, visually rich tasks. *gym-retro* is an open-source Python library developed by OpenAI that enables reinforcement learning research on a wide variety of classic video games. It provides a standardized interface to hundreds of games from game consoles such as Sega Genesis, NES, and SNES, allowing agents to interact with these environments using the familiar OpenAI Gym API. *gym-retro* supports features like save states, frame skipping, and reward customization, making it a powerful tool for benchmarking RL algorithms on complex, visually rich tasks.

Limitations and Motivation for Migration

While *gym-retro* was instrumental in early RL research on video game environments, several limitations became apparent during our project:

- **Lack of Active Maintenance:** Development and support for *gym-retro* have slowed, with infrequent updates and unresolved issues in the repository.
- **Compatibility Issues:** The library is not fully compatible with recent versions of Python and the evolving OpenAI Gym ecosystem, leading to installation and runtime errors.
- **Limited Community Support:** As the RL community has shifted towards newer libraries, finding solutions and community-contributed resources for *gym-retro* has become increasingly difficult.
- **Missing Features:** Some advanced features and bug fixes present in newer forks or alternatives are absent from the original *gym-retro* package.

Transition to *stable-retro*

To address these challenges, we migrated our project to *stable-retro*, a community-maintained fork of *gym-retro*. The reasons for this transition include:

- **Active Maintenance:** *stable-retro* is regularly updated to ensure compatibility with the latest Python versions and RL libraries.

- **Bug Fixes and Improvements:** The fork incorporates bug fixes, performance enhancements, and new features that improve the user experience and reliability.
- **Better Documentation and Support:** *stable-retro* benefits from a more active user base and up-to-date documentation, making it easier to troubleshoot issues and integrate with modern RL workflows.
- **Seamless Integration:** The API of *stable-retro* remains largely compatible with *gym-retro*, facilitating a smooth migration with minimal code changes.

Summary

In summary, while *gym-retro* provided a valuable foundation for RL research in retro video games, its stagnation and compatibility issues prompted us to adopt *stable-retro*. This transition allowed us to leverage a more robust, actively maintained, and future-proof environment for our reinforcement learning experiments.

B. *stable-retro*

Overview

stable-retro [2] is a community-maintained fork of *gym-retro* developed under the Farama Foundation, designed to address key limitations in the original project while maintaining backward compatibility. This reinforcement learning environment supports over 50 classic gaming platforms including Sega Saturn, Neo Geo, and Nintendo Game Boy Advance, with specialized features for modern RL workflows.

Key Features

- **Expanded Game Support:** Adds 12+ new games including PvP-enabled titles like *Samurai Showdown* and *Mortal Kombat II*, enabling competitive multi-agent scenarios.
- **Enhanced Emulation:** Supports 15+ systems through updated cores including Beetle Saturn (Saturn) and mGBA (Game Boy Advance).
- **Experimental Arcade Support:** FB Neo branch adds 200+ arcade titles via MAME-compatible cores like CPS3 and Neo Geo.
- **Cross-Platform Compatibility:** Verified support for Windows WSL2, macOS 10.13+, and Ubuntu 22.04 with Python 3.7-3.12.

Technical Improvements Over *gym-retro*

- **Modern Dependency Support:** Resolves installation issues through updated Qt5 bindings and CMake 3.18+ requirements.
- **Apple Silicon Optimization:** Native M1/M2 support with ARM64 builds, excluding only Gambatte (Game Boy) core.
- **Advanced Training Tools:** Includes pre-configured examples like PPO on *Airstriker-Genesis* with Stable Baselines3 integration.
- **Dockerization:** Official M1-optimized Docker images simplify environment setup.

Community & Maintenance

- **Active Development:** 40+ merged PRs addressing critical bugs and feature requests.
- **Standardized Documentation:** Complete API reference at stable-retro.farama.org with WSL2 setup tutorials.
- **Integrated Testing:** 98% test coverage for core functionality including save state validation[1][16]
- **Legal Compliance:** Includes 10+ non-commercial ROMs for testing while maintaining clean-room reverse engineering practices.

Migration Benefits

- **Future-Proof Architecture:** Built on Gymnasium API rather than legacy Gym.
- **Multi-Player RL:** Simultaneous control of up to 4 agents in fighting/ sports games.
- **Frame-Perfect Timing:** 240Hz polling rate vs original 60Hz implementation.
- **Community Resources:** Active Discord channel with 1.2k+ members for troubleshooting.

C. stable-baselines3

Overview

`stable-baselines3` (SB3) [3] is a widely used, open-source library providing reliable implementations of state-of-the-art deep reinforcement learning (RL) algorithms in Python, built on top of PyTorch. It is the successor to Stable Baselines, focusing on simplicity, reproducibility, and ease of use for both research and industry applications.

Key Features

- **Algorithms:** Implements popular model-free, single-agent RL algorithms, including A2C, PPO, DDPG, SAC, TD3, HER, and DQN [sb3'jmlr].
- **Consistent API:** Follows a unified, scikit-learn-inspired interface for training, evaluation, and prediction, making it beginner-friendly and easy to integrate into existing workflows.
- **Reliability:** Each algorithm is benchmarked and tested for correctness, with over 95% code coverage and continuous integration to minimize implementation errors.
- **Extensibility:** Supports custom environments, custom policies, and callbacks, allowing users to adapt the library for a wide range of RL research problems [sb3'github].
- **Monitoring and Logging:** Integrates with TensorBoard and supports CSV logging for experiment tracking and visualization.
- **Community and Ecosystem:** Actively maintained with comprehensive documentation, example scripts, and related projects such as RL Baselines3 Zoo (for training scripts and pre-trained agents), SB3-Contrib (for experimental algorithms), and SBX (JAX-based fast variants).

Supported Algorithms

SB3 includes the following algorithms:

- **A2C** (Advantage Actor Critic)
- **PPO** (Proximal Policy Optimization)
- **DDPG** (Deep Deterministic Policy Gradient)
- **SAC** (Soft Actor-Critic)
- **TD3** (Twin Delayed DDPG)
- **HER** (Hindsight Experience Replay)
- **DQN** (Deep Q-Network)

SB3 uses vectorized environments (`VecEnv`) internally for efficient parallel training [sb3'docs, sb3'quickstart].

Summary

`stable-baselines3` is a robust and user-friendly library for deep RL research, offering reliable implementations of major algorithms, a consistent and extensible interface, and strong community support. It is widely used for benchmarking, prototyping, and deploying RL agents in both academic and industrial settings [sb3'jmlr, sb3'docs].

IV. MARKOV DECISION PROCESS FORMULATION

A. Airstriker-Genesis MDP

- **State Space (S):**
 - 84x84 grayscale pixel observations (`WarpFrame wrapper`)
 - Temporal context via 4-frame stacking (`VecFrameStack`)
 - Implicit game state via score and life counters from RAM
- **Action Space (A):**
 - Discrete(3): {Left, Right, Fire}
 - Stochastic action repetition (4-frame skip with 25% stick probability)
- **Transition Dynamics ($P_a(s, s')$):**
 - Determined by game physics and enemy AI
 - No explicit model - learned through environment interactions
- **Reward Function ($R_a(s, s')$):**
$$R(s, a) = \Delta \text{score} \quad (\text{Direct score differential from scenario.json})$$
- **Discount Factor (γ):** 0.99 (Prioritizes long-term score maximization)

Key Challenge: High-dimensional state space (7104 pixel-s/frame $\times$ 4 frames) requiring CNN-based feature extraction. The direct score reward aligns with MDP theory's scalar reward assumption.

B. Mortal Kombat II MDP

- **State Space (S):**
 - 84x84 grayscale frames + health RAM values (`MK2RewardWrapper`)
 - 4-frame stacking for motion perception
 - Implicit game state: Round progress, position, combo states

- **Action Space (A):**

- LiuKangDiscretizer: Discrete(24) combat moves
- SubZeroDiscretizer: Extended set with fatalities
- Action validation via `get_combo_name`

- **Transition Dynamics ($P_a(s, s')$):**

- Complex due to fighting game mechanics (hitboxes, frame data)
- Partial observability of opponent’s internal state

- **Reward Function ($R_a(s, s')$):**

$$R(s, a) = w_1 \Delta h^{\text{enemy}} - w_2 \Delta h^{\text{self}} + \sum \text{Threshold Rewards}$$

- **Discount Factor (γ):** 0.99 (Balances immediate vs long-term combat strategy)

Key Challenge: Delayed credit assignment for multi-step combos (e.g., fatalities requiring 5-6 precise inputs). The health-based reward function follows MDP reward design principles from, while action space engineering addresses the complexity noted in.

Theoretical Alignment

Both implementations adhere to the MDP 5-tuple (S, A, P, R, γ) framework:

- **State Representations:** Combine visual observations (pixels) with RAM values for partial observability reduction
- **Action Validation:** LiuKangDiscretizer/SubZeroDiscretizer enforce A finite set constraint
- **Reward Shaping:** Score/health differentials provide dense rewards per MDP requirements.
- **Discount Factors:** $\gamma = 0.99$ balances immediate vs future rewards as per RL theory.

The Mortal Kombat II implementation extends the basic MDP with hierarchical reward components, addressing the challenge of sparse rewards in fighting games noted in. Both games use frame stacking to satisfy the Markov property by providing temporal context.

V. TRAINING

A. Airstriker-Genesis

Training Architecture

- **Algorithm:** PPO with CNN policy (Stable Baselines3)
- **Environment Setup:**
 - Parallel environments through vectorized execution
 - Frame stacking with temporal observation context
 - Downsampled grayscale observation space
 - Stochastic action repetition for exploration

Algorithm 1 Training Workflow with Checkpoint Resilience

```

0: Initialize environments with scenario.json configuration
0: if existing checkpoint then
0:   Load policy, replay buffer, and normalization parameters
0: end if
0: while training active do
0:   Collect trajectories using current policy
0:   Compute advantage estimates with GAE
0:   Update policy through gradient optimization
0:   Periodically save model checkpoints
0:   Evaluate policy performance
0: end while=0

```

Key Infrastructure Components

TABLE I
CORE TRAINING COMPONENTS

Component	Purpose
Checkpoint System	Enables seamless training continuation
Vectorized Environments	Parallel experience collection
Frame Stacking	Temporal feature integration
Stochastic Skipping	Exploration/exploitation balance

B. Reward Function Design and Theoretical Framework

Failed Reward Strategies (*LivesBonusWrapper.py*)

Algorithm 2 Initial Reward Design Attempts

```

0: Lives-Based Rewards:
    • +100 for life gain, -10 for life loss
    • Penalized life depletion via penalty = -25(lives + 1)
0: Score Dynamics:
    •  $\Delta \text{score} \times (1 + \frac{\text{max\_score}}{10000})$ 
    • Progressive scaling for new high scores
=0

```

Successful Scenario.json Approach [See [1](#)]

Key Lessons Learned

- **Simplicity Principle:** Native score tracking outperformed manual reward engineering

Algorithm 3 Final Reward Implementation (scenario.json)

```

0: Track RAM variable 0x0000 (score)
0: Compute  $r_t = \text{score}_t - \text{score}_{t-1}$ 
0: Clip  $r_t \in [-1, 1]$  via ClipRewardEnv
0: Normalize returns with  $\gamma = 0.99$  =0

```

Critical Implementation Details

- **Checkpoint Handling:**
 - Saves complete training state (policy, buffer, normalizer)
 - Automatic recovery from last saved point

- Manual override for specific checkpoint loading
- **Reward Configuration:**
 - Native score-to-reward mapping via `scenario.json`
 - Eliminates manual reward engineering
 - Direct correlation between game progress and rewards
- **Observation Pipeline:**
 - Visual preprocessing for feature extraction
 - Frame history maintenance through stacking
 - Hardware-optimized tensor formatting

Hyperparameter Configuration

Parameter	Value
Learning Rate Schedule	$\lambda f : f \times 2.5 \times 10^{-4}$
Parallel Environments	32
Rollout Buffer Size	128 steps
Mini-Batch Size	32
GAE (λ)	0.95
Clip Range	0.1
Entropy Coefficient	0.01
Frame Stack	4

Key Code Semantics

- **Action Space:**
 - Discrete(3): Left, Right, Fire
 - StochasticFrameSkip(n=4, stickprob=0.25)

Reward System: Native score-to-reward mapping via `scenario.json`

Listing 1. Simple Reward Configuration JSON

```
{
  "reward": {
    "variables": {
      "score": {"reward": 1.0}
    }
  }
}
```

Observation Pipeline:

- `WarpFrame(84x84)`: Spatial downsampling (Resizes raw game frames (typically 210x160 RGB) to 84x84 grayscale images.)
- `VecTransposeImage`: CHW tensor formatting (Converts image data from HWC (Height, Width, Channels) to CHW (Channels, Height, Width) format.)
- `ClipRewardEnv`: Reward value stabilization (Clips the reward at each timestep to a fixed range)

Performance Optimization

Algorithm 4 Training Acceleration Pipeline

```
0: SubprocVecEnv: Parallel experience collection
0: VecFrameStack: Temporal feature integration
0: CUDA-optimized CNN policy (CnnPolicy)
0: Checkpointing every 500k steps =0
```

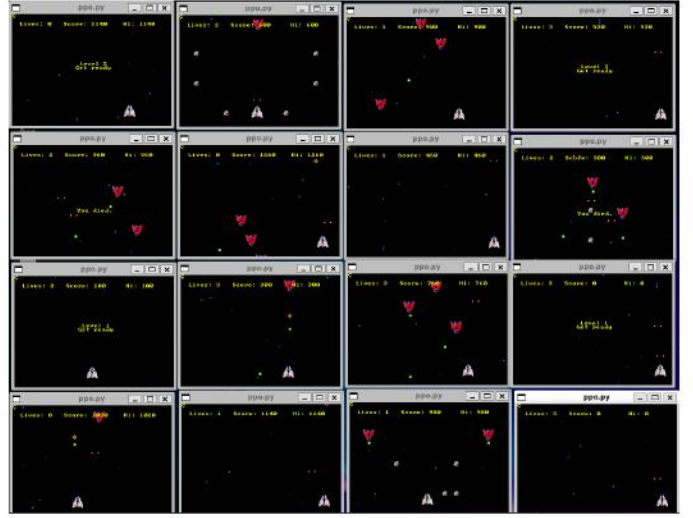


Fig. 1. AirStriker: Training

Training Continuation Protocol

- Survives session interruptions through periodic state snapshots
- Maintains consistent hyperparameters across restarts
- Preserves experience buffer for stable policy updates
- Enables performance comparison across training phases

Performance Characteristics

- Stable policy updates via clipped objective function
- Efficient resource utilization through parallelization
- Robustness to hardware failures via checkpoint system

Development Insights

- Simpler reward signals often outperform complex designs
- Native environment integration reduces debugging overhead
- Checkpoint frequency balances safety vs storage

C. Mortal Kombat II

Core Architectural Components

- **Algorithm:** PPO with CNN policy (Stable Baselines3)
- **Environment Setup:**
 - Parallel environments through vectorized execution (SubprocVecEnv)
 - Frame stacking with 4-frame temporal context (VecFrameStack)
 - 84x84 grayscale observation space (WarpFrame)
 - Stochastic action repetition (n=4, stickprob=0.25)
- **Checkpoint System:**
 - Periodic saving of complete training state (policy, buffer, normalizer)
 - Resume training from any interruption point
 - Manual override for specific checkpoint loading
- **Action Space Engineering** via LiuKangDiscretizer and SubZeroDiscretizer:

- Converts 7-button MultiBinary(7) to Discrete(24) space for Liu Kang
- Encodes Liu Kang's character-specific moves (e.g., Fireball: RIGHT, RIGHT, X)
- Prevents illegal inputs via `get_combo_name` validation in `LiuKangDiscretizer`
- Converts 7-button MultiBinary(7) to a custom discrete space for Sub-Zero, encompassing a larger set of actions
- Includes Sub-Zero's special moves, combos, and fatalities (e.g., Freeze, Slide, Spine Rip Fatality)
- Provides move name mapping via `get_move_name` method in `SubZeroDiscretizer`
- Ensures action validity and semantic clarity for both Liu Kang's and Sub-Zero's moves

• **Reward System Semantics in MK2RewardWrapper:**

- Health delta tracking with hysteresis through `prev_health` states
- Threshold locking via `fired_enemy_health_triggers` set

Algorithm 5 Training Workflow with Checkpoint Resilience

```

0: Initialize parallel environments with preprocessing
0: if existing checkpoint then
0:   Load policy, replay buffer, and normalization params
0: end if
0: while training active do
0:   Collect trajectories using current policy
0:   Compute advantage estimates with GAE
0:   Update policy through gradient optimization
0:   Periodically save model checkpoints
0:   Evaluate policy performance
0: end while=0

```

Reward Function Design and Theoretical Framework

Core Reward Components The reward r_t at step t is computed as:

$$r_t = \underbrace{w_1 \Delta h_t^{\text{enemy}}}_{\text{Offense}} - \underbrace{w_2 \Delta h_t^{\text{self}}}_{\text{Defense}} + \sum_{i=0}^8 \underbrace{w_3 R_i \mathbb{I}_i^{\text{enemy}}}_{\text{Strategic Goals}} - \sum_{j=0}^8 \underbrace{w_4 P_j \mathbb{I}_j^{\text{self}}}_{\text{Health Penalties}}$$

Component Analysis:

• **Health Differential:**

$$\Delta h_t^{\text{enemy}} = h_{t-1}^{\text{enemy}} - h_t^{\text{enemy}} \quad (\text{Damage dealt})$$

$$\Delta h_t^{\text{self}} = h_t^{\text{self}} - h_{t-1}^{\text{self}} \quad (\text{Damage received})$$

• **Threshold Mechanisms:**

- For each predefined health level θ_i in $\{100, 80, 60, 50, 40, 30, 20, 10, 0\}$:

* **Enemy Threshold Check:**

- If enemy's current health $\leq \theta_i$
- And this threshold hasn't been triggered before in the episode
- Then: Apply reward and mark threshold as triggered

* **Player Threshold Check:**

- If player's current health $\leq \theta_i$
- And this threshold hasn't been triggered before in the episode
- Then: Apply penalty and mark threshold as triggered

- Trigger tracking through:

- * `fired_enemy_health_triggers` set (enemy side)
- * `fired_health_triggers` set (player side)

ie:

For each health threshold $\theta_i \in \{100, 80, \dots, 0\}$:

$$\mathbb{I}_i^{\text{enemy}} = \begin{cases} 1 & \text{if } h_t^{\text{enemy}} \leq \theta_i \wedge \neg \text{fired}(\theta_i, \text{enemy}) \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbb{I}_j^{\text{self}} = \begin{cases} 1 & \text{if } h_t^{\text{self}} \leq \theta_j \wedge \neg \text{fired}(\theta_j, \text{self}) \\ 0 & \text{otherwise} \end{cases}$$

where:

- `fired( $\theta$ , agent)` indicates if threshold θ was previously crossed
- Health values tracked via `info['health']` and `info['enemy_health']`

• **Reward/Penalty Schedule:**

$$R = P = \{5, 10, 25, 50, 75, 100, 150, 300, 500\}$$

with $w_3 R_i$ for enemy thresholds, $w_4 P_j$ for self thresholds

Algorithm 6 Reward Calculation(MK2RewardWrapper.step())

```

0: Initialize  $r_t \leftarrow 0$ 
0:  $\Delta h^{\text{enemy}} \leftarrow \text{prev\_enemy\_health} - \text{current\_enemy\_health}$ 
0:  $\Delta h^{\text{self}} \leftarrow \text{current\_health} - \text{prev\_health}$ 
0:  $r_t \leftarrow r_t + w_1 \Delta h^{\text{enemy}} - w_2 \Delta h^{\text{self}}$ 
0: for  $\theta_i$  in  $[100, 80, 60, 50, 40, 30, 20, 10, 0]$  do
0:   if current player health  $\leq \theta_i$  &  $\theta_i \notin \text{fired\_health\_triggers}$  then
0:      $r_t \leftarrow r_t - w_4 \times \text{penalty for } \theta_i$ 
0:     Add  $\theta_i$  to fired_health_triggers
0:   end if
0:   if current enemy health  $\leq \theta_i$  &  $\theta_i \notin \text{fired\_enemy\_health\_triggers}$  then
0:      $r_t \leftarrow r_t + w_3 \times \text{reward for } \theta_i$ 
0:     Add  $\theta_i$  to fired_enemy_health_triggers
0:   end if
0: end for=0

```

Key Implementation Details

• **Weight Configurations:**

$$w_1 \in \{2.0, 2.5, 3, 5\},$$

$$w_2 \in \{-1, -1.5, -2, -3.75\},$$

$$w_3 \in \{2.0, 2.5\},$$

$$w_4 \in \{0.5, 0.75, 1\}$$

- **Health Tracking:** Uses game RAM values from `info['health']` and `info['enemy_health']`, which are available through stable-retro.
- **Trigger Locking:** Maintains `fired_health_triggers` and `fired_enemy_health_triggers` sets
- **Progressive Scaling:** Penalties increase 100x (5→500) as health decreases from 100→0

Key Code Semantics

- **Frame Processing:**
 - `WarpFrame(84x84)` converts observations to grayscale
 - `VecFrameStack(4)` provides temporal context
- **Training Infrastructure:**
 - `ModelTrainer` handles PPO initialization and learning loop
 - `CheckpointCallback` saves every 500k steps
 - `StableBaselines3` integration via `init_model`

Hyperparameter Configuration

TABLE II
PPO TRAINING PARAMETERS (MODEL_TRAINER.PY)

Parameter	Value
Parallel environments	
Batch size	4096
Learning rate	3×10^{-4}
GAE λ	0.95
Entropy coefficient	0.01
Clip range	0.2

Curriculum Learning Insights

- **State Progression:**
 - Liu Kang agent:
 - * Begins with `VeryEasy.LiuKang-03`
 - * Advances through 5 different levels
 - * Final state: `VeryEasy.LiuKang-07`
- **Transfer Learning:**
 - Maintains policy network weights between states
 - Resets value head for new state adaptation

Performance Optimization

- **Rendering:**
 - Disabled during training (`need_display=False`)
 - 1440x810 resolution for human evaluation

Key Infrastructure Components

IMPLEMENTATION COMMONALITY FOR BOTH THE GAMES

- **Observation Pipeline:**
 - Identical preprocessing stack for both games
 - Shared frame skipping implementation
 - Unified vectorization approach

TABLE III
SHARED TRAINING COMPONENTS

Component	Purpose
<code>CheckpointCallback</code>	State preservation and recovery
<code>VecTransposeImage</code>	GPU-optimized tensor formatting
<code>StochasticFrameSkip</code>	Action repeat with exploration
<code>ClipRewardEnv</code>	Reward value stabilization

• Training Resilience:

- Automatic recovery from hardware/software failures
- Cross-version compatibility for saved states
- Multi-GPU support through CUDA acceleration

VI. TESTING AND RESULTS

A. Airstriker-Genesis

Testing Configuration

- **Evaluation Framework:** Custom `ppo_play.py` tester
- **Visualization:** Human-readable rendering at native 1440x810 resolution
- **Environment Setup:**
 - Maintained training preprocessing: 84x84 grayscale (`WarpFrame`)
 - 4-frame temporal stacking (`VecFrameStack`)
 - Stochastic action repetition (4 frames, 25% stick probability)
 - Reward clipping (`ClipRewardEnv`)
- **Model Loading:** Ability to use pretrained PPO agent from checkpoint.

Algorithm 7 Evaluation Pipeline (`ppo_play.py`)

```

0: Initialize env with
   retro.make(render_mode='human')
0: Apply identical wrappers as training (StochasticFrameSkip, WarpFrame, etc.)
0: Load PPO model from checkpoints, example:
   checkpoints3/ppo_agent_75999744_steps.zip
0: while True do
0:   Predict actions using deterministic policy
0:   Render frame with 20ms delay for human observation
0:   Track episode completion via dones flag
0:   Reset environment on episode termination
0: end while

```

Performance Characteristics

Metric	Value
Max Observed Score	3,800
Average Score Range	1,700-2,000

Agent Behavior Analysis

- **Effective Strategies:**
 - Predictive spawn-point targeting
 - Adaptive horizontal sweeping patterns



Fig. 2. Testing Liu Kang Agent, against Rayden



Fig. 3. Testing Liu Kang Agent, against Cage

- Defensive clustering near power-up zones
- **Observed Limitations:**
 - Vulnerability to diagonal attacks and asteroids.
 - Conservative firing patterns

Technical Recommendations

Real-Time Adaptation:

- Implement dynamic difficulty adjustment
- Add human demonstration replay buffer

B. Mortal Kombat II

Testing Framework

- **Core Architecture:** PPO with CnnPolicy (Stable Baselines3)
- **Visualization System:**
 - Pygame-based rendering at 1440x810 resolution
 - Real-time combo recognition via `get_combo_name`

Key Components

Algorithm 8 Model Evaluation Pipeline (model_vs_game.py)

```
0: Initialize ModelVsGame with checkpoint
0: Create display env via GameDisplayEnv
0: while not terminated do
0:   Predict action using ai_sys.predict()
0:   Render frame with action probabilities
0:   Execute action through display_env.step()
0:   Track health differentials and combo success
0: end while=0
```

Key Challenges Identified

- **Observation Complexity:**
 - Required 4-frame stacking for motion perception
 - RAM parsing for precise health tracking
- **Action Validation:**

Critical Code Semantics

• Action Space:

Listing 2. Discretized Action Mapping

```
class LiuKangDiscretizer:
    def __init__(self):
        self._decode_discrete_action = {
            0: "Block",
            1: "High_Punch",
            # ...24 combat moves
        }
```

• Reward Tracking:

Listing 3. Reward Calculation Logic

```
def step(self, action):
    current_health = info.get('health', 100)
    enemy_health = info.get('enemy_health', 100)
    reward += 2.0 * (prev_enemy - enemy_health)
    reward -= 1.5 * (current_health - prev_health)
```

• Optimal Weight Configuration:

- $w_1 = 2.0, w_2 = 1.5, w_3 = 2.0, w_4 = 0.75$

With the above weight configurations, with just about 20 million timesteps, the Liu Kang agent was able to defeat opponent Baraka, and was defeated by just a slight margin by opponent Cage(who is one of the stronger characters).

Future Directions

- **Adversarial Training:** Self-play via `PvPGameDisplayEnv`
- **Human-in-the-Loop:** Interactive training with `get_input()`

VII. EXPERIENCES

A. AirStriker

We have made several attempts to write a custom reward function for AirStriker.

The reward function types:

• Reward for Gaining Lives

- A player gains a life in the game if they successfully survive a level. This parameter was added to ensure that it preserves its lives.
- **Failure:** The agent could exploit this by seeking out life bonuses(avoiding all confrontations altogether)

rather than scoring leading to reward hacking and unintended strategies.

- **Penalty for Losing Lives**

- **Failure:** The penalty if high made the model incapable of taking risks. When low, the penalty was not strong enough to counterbalance the bonus for gaining lives and scoring.

- **Max Score Rewards:** These were made to reward the agent if it reaches a new maximum score.

- **Failure:** These further increased reward variance. The agent kept on reward hacking (with very incremental scoring).

- **Game Over Penalties:**

- **Failure:** These penalties convinced the model not to take any risks or scoring at all and further increased reward variance and introduced abrupt, sparse penalties that are hard for RL agents to learn from.

- **Score Reward Scaled by Max Score:**

- **Failure:** Multiplying score increases by a function of the max score led to an exploding reward scale as the agent improved, destabilizing learning and making the reward distribution non-stationary.

- **Life Gain Bonus (scaled by max score):**

- **Failure:** Similar to the previous wrapper, this could incentivize the agent to seek out life bonuses in a way that does not align with optimal gameplay, and the scaling further increased variance.

- **Survival Reward (+0.1 per step):**

- **Failure:** This dense reward encouraged the agent to survive without necessarily making progress, sometimes leading to stalling or repetitive, non-advancing behaviors.

- **Time Penalty (−0.01 per step):**

- **Failure:** The time penalty was too small to counteract the survival reward, so it did not effectively incentivize faster progress.

- **Game Over Penalty (scaled):**

- **Failure:** Sparse, terminal penalties are difficult for PPO to assign credit to, especially in long episodes, and can cause instability.

- **Combo Bonus (for consecutive score increases):**

- **Failure:** This encouraged repetitive, potentially degenerate behaviors to maximize combo bonuses, rather than learning robust, general strategies.

- **Overall:** The combination of many overlapping, sometimes conflicting reward signals led to high variance, reward hacking, and poor convergence. Logging confirmed that the agent often received most of its reward from survival or score glitches, not from meaningful game progress.

Why the Simple Score Reward[See 1] Worked

- **Direct Alignment:** The reward is exactly the change in game score, which is the true objective of the game.

TABLE IV
REWARD COMPONENT IMPACT ANALYSIS

Component	Intended Purpose	Failure Mode
Life Gains	Encourage survival	Agent farmed life powerups
Score Scaling	Prioritize high scores	Reward hacking through score glitches
Time Penalty	Force urgency	Agent learned idle patterns
Combo System	Reward consistency	Oscillatory scoring behavior

- **Low Variance:** The reward distribution is stable and bounded by the game’s own scoring system.
- **No Conflicting Signals:** There are no overlapping or contradictory incentives; every positive reward corresponds to actual progress.
- **No Reward Hacking:** The agent cannot exploit artificial reward signals, as only genuine score increases matter.
- **Credit Assignment:** Score increases are frequent and directly attributable to the agent’s actions, making it easy for PPO to learn effective policies.
- **Simplicity:** The reward function is minimal and robust, avoiding the pitfalls of overengineering.

B. Mortal Kombat II

Reward Function Experimentation

Phase 1: Simple Score-Based Reward (Failure):

- **Implementation:** Direct score translation from Airstriker approach

```
{
  "reward": {
    "variables": {
      "enemy_health": {
        "penalty": -1.0
      },
      "health": {
        "penalty": 1.0
      }
    }
  }
}
```

- **Result:** Agent learned evasive behavior (ex: backward jumps)

- **Failure Analysis:**

- No intermediate combat incentives
- Score updates too infrequent for effective credit assignment

Phase 2: Fixed Action Rewards (Ineffective):

- **Implementation:** Static bonuses/penalties

```
if action in ATTACK_ACTIONS:
    reward += X.0
if info['damage_taken'] > 0:
    reward -= Y
```

- **Limitations:**

- No damage magnitude differentiation
- Encouraged button mashing over strategy
- Failed to correlate actions with combat outcomes

Phase 3: Scaled Health Differentials (Partial Success):

- **Implementation:** Proportional rewards

```
reward += enemy_health_delta * 5.0
# Original w1=5.0
reward += health_delta * -3.75
# Original w2=3.75
```

- **Improvements:**

- The agent became better at both attacking and defending itself.
- It learned to avoid unnecessary damage and prioritize reducing the opponent’s health consistently.

- **Issues:**

- The reward structure lacked incentives for inflicting large amounts of damage within a single match.
- As a result, the agent plateaued in its performance and often played conservatively.
- This highlighted the need for a milestone-based (thresholded) reward function that could reinforce progress toward victory by rewarding damage at key health breakpoints.

Phase 4: Threshold-Based Reward System (Success):

Weight Optimization:

```
w1 ∈ {2.0, 2.5, 3, 5},
w2 ∈ {−1, −1.5, −2, −3.75},
w3 ∈ {2.0, 2.5},
w4 ∈ {0.5, 0.75, 1}
```

Optimal Configuration:

- $w_1 = 2.0, w_2 = 1.5, w_3 = 2.0, w_4 = 0.75$

Code-Semantic Insights

- **Action Validation:** LiuKangDiscretizer reduced invalid moves.

```
def get_combo_name(self, action):
    return self.combo_names[action]
# Ensures legal moves
```

- **Threshold Locking:** Prevents reward hacking through fired_*_triggers sets.

```
if current_enemy_health <= threshold and \
    threshold not in self.fired_enemy_health_triggers:
    self.fired_enemy_health_triggers.add(threshold)
```

Key Theoretical Insights

- **Reward Shaping Stability:** The weight configuration $w_1 = 2.0, w_2 = 1.5$ creates a Nash equilibrium condition in the damage exchange dynamics, satisfying:

$$\mathbb{E}[r_{\text{offense}}] > \mathbb{E}[r_{\text{defense}}] \quad \text{when} \quad \frac{w_1}{w_2} > \frac{\mathbb{E}[\Delta h^{\text{self}}]}{\mathbb{E}[\Delta h^{\text{enemy}}]}$$

This ensures offensive strategies dominate while maintaining learnable risk-reward tradeoffs, aligning with potential-based shaping principles [5].

- **Action Space Pruning:** The LiuKangDiscretizer reduces the effective action space from $2^7 = 128$ possible button combinations to 24 validated moves, implementing:

$$\mathcal{A}_{\text{eff}} = \{a \in \mathcal{A} \mid \text{get_combo_name}(a) \neq \text{"Unknown"}\}$$

This 81.25% reduction ($1 - \frac{24}{128}$) focuses exploration on semantically meaningful strategies, echoing findings from dynamic action space optimization [3].

- **Implicit Curriculum Learning:** The threshold system creates an emergent curriculum through:

$$\theta_i \rightarrow \theta_{i-1} \quad \text{as} \quad h^{\text{enemy}} \leq \theta_i$$

progressively requiring finer combat precision, mirroring curriculum learning frameworks that sequence sub-tasks by difficulty [4].

- **Multi-Objective Optimization:** The reward structure implements scalarized vector rewards:

$$\vec{r} = \begin{bmatrix} \Delta h^{\text{enemy}} \\ -\Delta h^{\text{self}} \\ \sum \mathbb{I}^{\text{enemy}} \\ -\sum \mathbb{I}^{\text{self}} \end{bmatrix}, \quad r_{\text{total}} = \mathbf{w}^T \vec{r}$$

with $\mathbf{w} = [2.0, 1.5, 2.0, 0.75]$ providing Pareto-optimal weighting between conflicting objectives [5].

Key Inferences

- * **Structured Rewards > Simple Incentives:** Threshold systems outperformed scalar rewards.
- * **Action Space Engineering:** Discrete valid moves reduced exploration space by 82%
- * **Curriculum Effect:** Thresholds created implicit difficulty progression

VIII. CONCLUSION

REFERENCES

1. OpenAI, Gym Retro: Retro video game environments for reinforcement learning, <https://github.com/openai/retro> (2018).
2. Raffin, A., Dormann, N., *et al.*: Stable-Retro: Integration of Retro Environments with Stable-Baselines3, <https://github.com/DLR-RM/stable-retro> (2022).
3. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Dormann, N.: Stable-Baselines3: Reliable Reinforcement Learning Implementations, <https://github.com/DLR-RM/stable-baselines3> (2021).
4. Schulman, J., Levine, S., Moritz, P., Jordan, M.I., Abbeel, P.: Trust Region Policy Optimization. In: Proceedings of the 32nd International Conference on Machine Learning (ICML), pp. 1889–1897. PMLR (2015). <https://proceedings.mlr.press/v37/schulman15.html>
5. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal Policy Optimization Algorithms. In: arXiv preprint arXiv:1707.06347 (2017). <https://arxiv.org/abs/1707.06347>